BÖLÜM 5 / MODULE 5

İleri Düzey RAG (Advanced RAG) & Bilgi Getirme Optimizasyonu

Advanced RAG & Information Retrieval Optimization

Hibrit Arama (Dense Vector + Sparse BM25) ve Reciprocal Rank Fusion (RRF), Cross-Encoder Re-ranking (Cohere), Sorgu Dönüşümü (HyDE & Multi-Query), Lost in the Middle problemi ve RAG Triad (RAGAS) değerlendirme metrikleri.

Hybrid Search (Dense + Sparse BM25), Reciprocal Rank Fusion (RRF), Cross-Encoder Re-ranking, Query Transformation (HyDE & Multi-Query), Context Compression, and automated RAG Triad (RAGAS) evaluation frameworks.

25

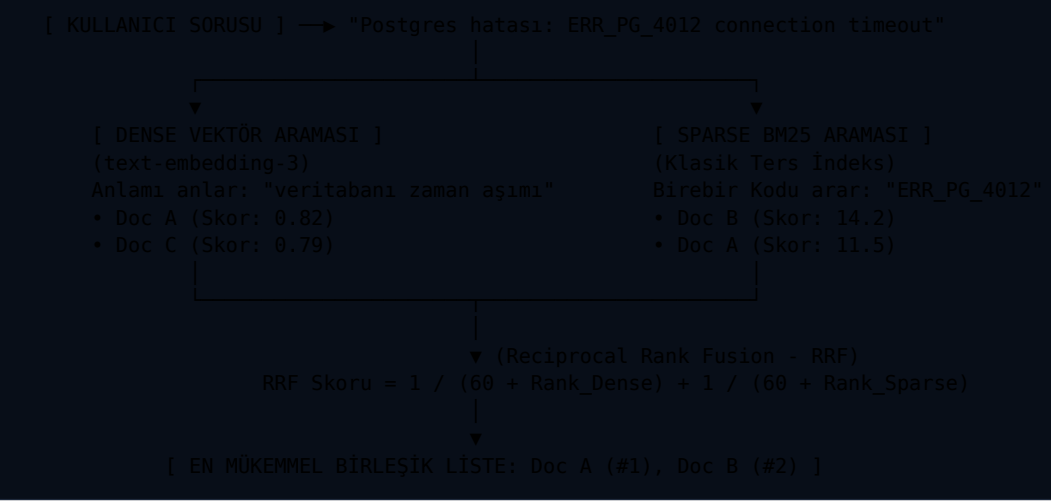
BÖLÜM: İLERI DÜZEY RAG (ADVANCED RAG) & BILGI GETIRME OPTIMIZASYONU | MODULE: ADVANCED RAG & INFORMATION RETRIEVAL OPTIMIZATION

Hibrit Arama (Hybrid Search): Vektör (Dense) + BM25 (Sparse) ile Reciprocal Rank Fusion

Hybrid Search: Combining Dense Vector Embeddings with Sparse BM25 via RRF

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir dedektifin tek bir yardımcısı olursa işler aksayabilir. Biri felsefeden ve hislerden anlayan bir psikolog olsun (Vektör / Dense Search: 'Kullanıcı veritabanı kilitlenmesinden bahsediyor'). Diğeri ise sadece plaka ve kimlik numarası ezberleyen bir parmak izi uzmanı olsun (BM25 / Sparse Search: 'Aranan hata kodu tam olarak ERR_PG_4012!'). İkisi bir araya geldiğinde (Hibrit Arama), psikolog konunun özünü anlarken parmak izi uzmanı tam nokta atışı hata kodunu yakalar! Tek başına vektör araması hata kodlarını ve seri numaralarını kaçırır; BM25 ile birleşince yenilmez olur!

🇬🇧 ENGLISH (ELI5)

Imagine a murder investigation with two specialized detectives: Detective Vector evaluates psychological motive and semantic themes ('The killer felt betrayed'), while Detective BM25 searches for exact fingerprint serial numbers ('Weapon registered to ID 9481'). Pure vector search excels at high-level concepts but stumbles on exact alphanumeric product codes, API tokens, and error strings. Hybrid Search fuses Dense Vector embeddings with Sparse BM25 keyword matching via Reciprocal Rank Fusion (RRF), capturing both semantic nuance and exact lexical matches!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Hibrit Arama ve RRF Matematiği: 1) Dense Vector (Yoğun Vektör): 1536 float sayısıdır; anlam benzerliğini yakalar ama 'iPhone 15 Pro Max 256GB' ile 'iPhone 14 Pro Max 256GB' arasındaki minik rakam farkını kaçırabilir. 2) Sparse Vector (BM25 - Seyrek Vektör): TF-IDF'in gelişmiş halidir; nadir kelimelere yüksek ağırlık verir, exact match'i yakalar. 3) Reciprocal Rank Fusion (RRF): Farklı ölçeklerdeki skorları (Cosine 0-1 ile BM25 0-50) doğrudan toplayamazsınız! RRF skorları değil, sıralama sıralarını (Rank) birleştirir: RRF(d) = Σ [1 / (k + rank_m(d))]. Genellikle k = 60 sabiti kullanılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Hybrid Search Topology & Reciprocal Rank Fusion (RRF): 1) Dense Vectors: Project text into continuous semantic manifolds. Prone to precision collapse on exact alphanumeric SKU tokens, legal clauses, and stack traces. 2) Sparse BM25 Vectors: Exact lexical term frequency-inverse document frequency weighting. 3) Normalization Impedance: Raw cosine scores (0.0 to 1.0) cannot be naively added to unbounded BM25 scores (0 to 45). RRF solves this by operating strictly on ordinal rank positions: RRF(d) = Σ [1 / (k + rank_m(d))], where k=60 dampens the influence of top outliers across heterogeneous result lists.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: Reciprocal Rank Fusion (RRF) Algoritması ile Liste Birleştirme
import java.util.*;

public class ReciprocalRankFusion {
    private static final int K = 60; // Standart RRF yumuşatma sabiti

    public record SearchResult(String docId, double score) {}

    public static List<SearchResult> mergeRrf(List<String> denseRankedIds, List<String>
sparseRankedIds, int topN) {
        Map<String, Double> rrfScores = new HashMap<>();

        // 1. Dense sıralamasını işle
        for (int rank = 0; rank < denseRankedIds.size(); rank++) {
            String docId = denseRankedIds.get(rank);
            rrfScores.put(docId, rrfScores.getOrDefault(docId, 0.0) + (1.0 / (K + rank + 1)));
        }

        // 2. Sparse (BM25) sıralamasını işle
        for (int rank = 0; rank < sparseRankedIds.size(); rank++) {
            String docId = sparseRankedIds.get(rank);
            rrfScores.put(docId, rrfScores.getOrDefault(docId, 0.0) + (1.0 / (K + rank + 1)));
        }

        // 3. Skorlara göre azalan sırada sırala ve ilk TopN sonucu al
        return rrfScores.entrySet().stream()
            .sorted(Map.Entry.<String, Double>comparingByValue().reversed())
            .limit(topN)
            .map(e -> new SearchResult(e.getKey(), e.getValue()))
            .toList();
    }
}

void main() {
    List<String> vectorResults = List.of("DocA", "DocB", "DocC");
    List<String> bm25Results   = List.of("DocB", "DocA", "DocD");

    var combined = ReciprocalRankFusion.mergeRrf(vectorResults, bm25Results, 3);
    for (var res : combined) {
        IO.println("Doküman: " + res.docId() + " | RRF Skoru: " + String.format("%.5f", res.score()));
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Mülakat Sorusu: 'Neden Cosine benzerlik skoru ile BM25 skorunu doğrudan toplayıp ortalamasını almıyoruz?' Cevap: 'Çünkü iki algoritmanın skor dağılımları ve varyansları tamamen farklıdır (Normalizasyon Uyuşmazlığı). Doğrudan toplamak bir metriğin diğerini ezmesine yol açar; bu yüzden rütbe tabanlı RRF zorunludur.'

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Interview trap: 'Why not simply average the Cosine and BM25 scores together?' Decisive answer: 'Cosine and BM25 follow incompatible statistical probability distributions. Raw score summation leads to one distribution dominating the other. Rank-based RRF is scale-invariant and immune to distribution skew.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Hibrit arama (Dense + Sparse), kurumsal RAG'de bilgi getirme doğruluğunu %30 artırır.
- BM25 exact match (hata kodları, seri no) için; Vektör ise kavramsal anlam için şarttır.
- Reciprocal Rank Fusion (RRF), farklı arama motorlarının sonuçlarını adilce birleştirir.

🎯 Key Takeaways for the Interview

- Hybrid search delivers a 20-30% recall boost over pure dense vector retrieval in enterprise domains.
- BM25 guarantees exact keyword hits; dense vectors provide semantic generalization.
- RRF provides a mathematically robust, scale-invariant rank-fusion algorithm.

26

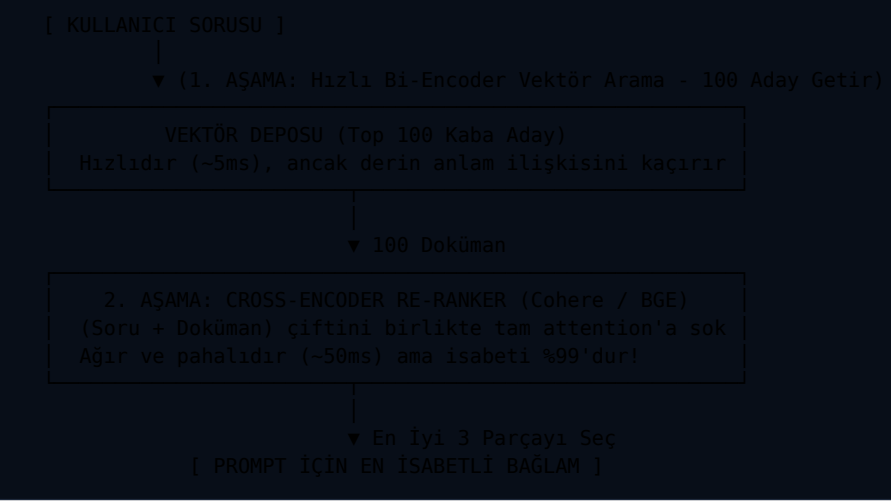
BÖLÜM: İLERI DÜZEY RAG (ADVANCED RAG) & BILGI GETIRME OPTIMIZASYONU | MODULE: ADVANCED RAG & INFORMATION RETRIEVAL OPTIMIZATION

Yeniden Sıralama (Re-ranking): Cross-Encoder Modelleri ve Cohere Rerank

Two-Stage Retrieval: Cross-Encoders & Re-ranking with Cohere / BGE-Reranker

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirkete 10.000 kişi iş başvurusu yaptığında genel müdür 10.000 CV'nin hepsini tek tek 1 saat boyunca mülakat edemez (Cross-Encoder çok yavaştır). Önce İK bir filtre koyar: 'Mühendislik mezunu ilk 100 kişiyi ayır' (1. Aşama: Hızlı Vektör Arama). Sonra genel müdür bu 100 adayı masasına alır, her birinin gözünün içine bakarak detaylı mülakat yapar ve en iyi 3 kişiyi işe alır (2. Aşama: Re-ranking)! Re-ranking, arama motorunun getirdiği 50 parçayı tekrar okuyup en doğru ilk 3 parçayı öne çıkarma sanatıdır!

🇬🇧 ENGLISH (ELI5)

When 10,000 candidates apply for an engineering job, the VP of Engineering cannot conduct a 60-minute deep technical interview with all 10,000 (Cross-Encoder is computationally heavy). First, HR runs an automated keyword/vector filter to select the top 50 resumes in seconds (Stage 1: Bi-Encoder ANN Search). Then, the VP personally evaluates those 50 resumes with rigorous scrutiny to select the 3 best hires (Stage 2: Re-ranking). Two-stage retrieval pairs fast vector search with cross-encoder precision!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Bi-Encoder vs Cross-Encoder Mimarisi: 1) Bi-Encoder (Standart Embedding): Dokümanı ve sorguyu birbirinden bağımsız embed eder. Hızlıdır (O(1) karşılaştırma) ancak soru ile metin arasındaki çapraz dikkat (Cross-Attention) kurulamaz. 2) Cross-Encoder (Reranker): `(Sorgu, Doküman)` çiftini tek bir transformer modeline aynı anda sokar. Token'ların birbirine olan ilgisini tam olarak hesaplar. 1000 dokümanda çok yavaştır, ancak ilk 50 dokümanı yeniden sıralamak için idealdir (ör. Cohere Rerank, BGE-Reranker-v2). 3) Metrik Kazancı: Re-ranking aşaması RAG sistemlerinde NDCG@10 ve MRR metriklerini %25'e kadar yükseltir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Bi-Encoder vs Cross-Encoder Computational Topologies: 1) Bi-Encoders: Map query and document independently into fixed-length vectors (f(q) and f(d)). Search relies on simple inner products (f(q) · f(d)). Extremely fast, but zero cross-token interaction. 2) Cross-Encoders: Ingest query and document concatenated as a single sequence ([CLS] ∘ q ∘ [SEP] ∘ d). Every query token attends directly to every document token across all transformer layers. Prohibitively expensive across millions of documents, but unbeatable when re-scoring the top 20-50 candidates. 3) Impact: Typically elevates MRR@10 and NDCG@5 by 15-25% in enterprise benchmarks.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & LangChain4j: Cohere / ScoringModel ile Re-ranking Entegrasyonu
import dev.langchain4j.data.segment.TextSegment;
import dev.langchain4j.model.cohere.CohereScoringModel;
import dev.langchain4j.rag.query.Query;
import java.util.List;

public class ReRankingPipeline {
    private final CohereScoringModel scoringModel;

    public ReRankingPipeline(String cohereApiKey) {
        this.scoringModel = CohereScoringModel.builder()
            .apiKey(cohereApiKey)
            .modelName("rerank-v3.5")
            .build();
    }

    public List<TextSegment> reRankCandidates(String userQuery, List<TextSegment> initialCandidates,
int topN) {
        var query = Query.from(userQuery);

        // Cross-encoder ile adayları puanla
        var scoredList = scoringModel.scoreAll(initialCandidates, userQuery).content();

        // En yüksek puana göre sırala ve ilk topN'i döndür
        return initialCandidates.stream()
            .sorted(Comparator.comparingDouble((TextSegment seg) ->
                scoredList.get(initialCandidates.indexOf(seg))).reversed())
            .limit(topN)
            .toList();
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Gecikme Uyarısı (Latency Budget): Re-ranking aşaması sisteme ek bir 50-150ms gecikme ekler. Eğer uygulamanız SLA olarak <300ms yanıt süresi hedefliyorsa, re-ranker'a 100 doküman yerine sadece en iyi 15-20 dokümanı gönderin.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Latency budget hazard: Cross-encoder re-ranking adds 50-150ms of network and inference latency. If your production SLA targets sub-300ms total response time, cap your candidate pool to top 15-20 segments before reranking.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- İki aşamalı getirme (Two-Stage Retrieval), hız ile doğruluğu mükemmel dengeler.
- Bi-Encoder ilk elemede hızlı arama yapar; Cross-Encoder en iyi adayları kesin sıralar.
- Cohere Rerank veya yerel BGE-Reranker, RAG halüsinasyonlarını doğrudan azaltır.

🎯 Key Takeaways for the Interview

- Two-stage retrieval delivers the optimal compromise between low latency and high accuracy.
- Bi-encoders prune millions of candidates; cross-encoders refine the top candidates.
- Re-ranking significantly reduces hallucination risk by boosting true relevant context to the top.

27

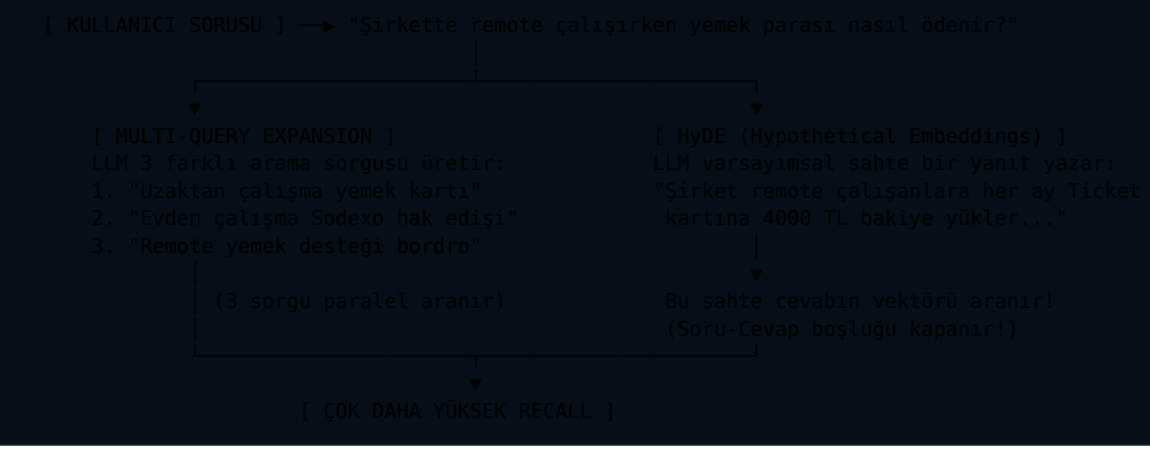
BÖLÜM: İLERI DÜZEY RAG (ADVANCED RAG) & BILGI GETIRME OPTIMIZASYONU | MODULE: ADVANCED RAG & INFORMATION RETRIEVAL OPTIMIZATION

Sorgu Dönüşümü: Query Expansion, Multi-Query & HyDE (Hypothetical Document Embeddings)

Query Transformation: Query Expansion, Multi-Query & HyDE (Hypothetical Embeddings)

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir doktorun odasına girip sadece 'Karnım ağrıyor' derseniz, doktor koca tıp kitabında nereye bakacağını şaşırabilir. Akıllı bir doktor soruyu kafasında genişletir: 'Apandisit mi, mide spazmı mı, zehirlenme mi?' (Multi-Query). Ya da daha da iyisi, doktor kafasında hayali bir hasta raporu canlandırır: 'Hasta dün deniz mahsulü yemişse gıda zehirlenmesidir' (HyDE) ve o hayali cevaba uyan tıbbi makaleleri arar! HyDE tam olarak budur: Sorunun değil, 'olması muhtemel hayali cevabın' vektörünü aratarak arama isabetini tavan yaptırır!

🇬🇧 ENGLISH (ELI5)

When a patient walks into a clinic and says 'My stomach hurts', searching a medical textbook for those exact 3 words is useless. An experienced physician expands the query into distinct clinical hypotheses: 'Appendicitis, acute gastritis, foodborne pathogen' (Multi-Query). Even better, the physician hallucinates a hypothetical clinical outcome: 'Patient consumed raw shellfish, exhibiting acute bacterial gastroenteritis' (HyDE) and searches the database using that hypothetical text! Hypothetical Document Embeddings (HyDE) bridges the semantic gap between questions and answers!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Sorgu Dönüşüm Stratejileri: 1) Multi-Query: LLM kullanıcının tek bir sorgusundan 3-5 farklı eşanlamlı sorgu üretir. Tüm sorgular paralel (Java 25 Virtual Threads ile) vektör deposunda taranır ve sonuçlar birleştirilir. 2) HyDE (Gao et al., 2022): 'Soru ile Cevap' vektör uzayında aynı yerde durmaz (Asimetrik Arama Problemi). HyDE önce LLM'e 'Bu sorunun cevabı sence nasıl görünürdü?' diye sahte bir paragraf yazdırır. Bu sahte paragrafın vektörü, gerçek dokümanların vektörüne sorunun kendisinden %40 daha yakındır! 3) Query Decomposition: Karmaşık soruları alt sorulara bölme.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Advanced Query Transformation Mechanics: 1) Multi-Query Generation: LLMs synthesize 3-5 paraphrased permutations to overcome lexical brittleness. Concurrent retrieval via Java 25 Virtual Threads ensures zero wall-clock penalty. 2) HyDE (Hypothetical Document Embeddings): Addresses the Asymmetric Retrieval Problem: questions and answers inhabit distinct topological clusters in vector space. HyDE prompts an LLM to generate an idealized hypothetical answer. Even if factually inaccurate, its vector manifold aligns closely with true candidate answers, boosting zero-shot recall. 3) Query Decomposition: Deconstructs multi-hop prompts into sub-queries.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & Spring AI: HyDE (Hypothetical Document Embedding) Servisi
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.document.Document;
import org.springframework.ai.vectorstore.SearchRequest;
import org.springframework.ai.vectorstore.VectorStore;
import java.util.List;

public class HydeRetrievalService {
    private final ChatClient chatClient;
    private final VectorStore vectorStore;

    public HydeRetrievalService(ChatClient.Builder builder, VectorStore vectorStore) {
        this.chatClient = builder.build();
        this.vectorStore = vectorStore;
    }

    public List<Document> searchWithHyde(String userQuestion) {
        // 1. LLM'e varsayımsal (hypothetical) bir cevap yazdır
        String hypotheticalAnswer = chatClient.prompt()
            .system("Verilen soru için teknik ve kurumsal bir dokümanda yer alabilecek olası cevabı
yaz.")
            .user(userQuestion)
            .call()
            .content();

        // 2. Soruyu değil, varsayımsal cevabın embedding'ini arat!
        return vectorStore.similaritySearch(
            SearchRequest.builder()
                .query(hypotheticalAnswer)
                .topK(3)
                .similarityThreshold(0.70)
                .build()
        );
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Gecikme & Maliyet Dengesi: HyDE ekstra bir LLM çağrısı gerektirir (yaklaşık +400ms ve ek token maliyeti). Her basit soru için HyDE çalıştırmak israftır. Yalnızca kaba vektör aramasının skorunun <0.60 kaldığı durumlarda Fallback olarak tetiklenmelidir.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Cost & latency trade-off: HyDE incurs an upfront LLM completion call (+400ms and inference tokens). Never execute HyDE unconditionally on trivial queries; trigger it conditionally only when initial ANN retrieval confidence scores fall below threshold.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Soru ile cevap vektör uzayında farklı yönlere bakabilir (Asimetrik arama).
- HyDE, hayali bir cevap üreterek vektör aramasını cevabın diline yaklaştırır.
- Multi-Query, farklı arama kelimelerini paralel tarayarak doküman kaçırma riskini bitirir.

🎯 Key Takeaways for the Interview

- Questions and answers naturally occupy distinct vector clusters (Asymmetric Retrieval).
- HyDE bridges the question-answer divide by searching with hypothetical answer embeddings.
- Multi-Query eliminates lexical brittleness by retrieving across diverse paraphrased queries.

28

BÖLÜM: İLERI DÜZEY RAG (ADVANCED RAG) & BILGI GETIRME OPTIMIZASYONU | MODULE: ADVANCED RAG & INFORMATION RETRIEVAL OPTIMIZATION

Bağlam Sıkıştırma (Context Compression) & 'Lost in the Middle' Problemi

Context Compression & Mitigating the 'Lost in the Middle' Attention Deficit

🤖 AI Mimarisi & Veri Akış Şeması 🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Öğretmeniniz size 50 sayfalık bir kitap verip 'Yarın sınav var' dediğinde ne yaparsınız? Kitabın ilk sayfasını dikkatle okursunuz (Aklınızda kalır). Son sayfasını da yeni okuduğunuz için hatırlarsınız. Ama 25. sayfadaki o küçük detayı kesinlikle unutursunuz! Yapay zeka modelleri de tıpkı insanlar gibi dikkat dağınıklığı yaşar (Lost in the Middle): Bağlam uzadıkça ortada duran bilgileri görmezden gelir. Bağlam sıkıştırma ise kitaptaki tüm gereksiz 've, veya, çünkü' kelimelerini atıp sadece sınavda çıkacak 3 cümleyi modelin tam gözünün önüne koymaktır!

🇬🇧 ENGLISH (ELI5)

When crammed with a 200-page dossier before a trial, a human attorney vividly recalls the opening argument (Primacy Effect) and the final conclusion (Recency Effect), but glazes over fine print buried on page 97. Transformer attention mechanisms suffer from the identical cognitive bias: 'Lost in the Middle' (Liu et al., Stanford/Berkeley). Model recall plummets when critical facts reside in the middle of long contexts. Context Compression strips conversational fluff, while Context Reordering places the highest-ranked evidence at the extreme boundaries!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dikkat Mekanizması ve Sıkıştırma Teknikleri: 1) Lost in the Middle Fenomeni: U-şeklinde bir doğruluk eğrisidir. Pozisyonel embedding'ler (RoPE / ALiBi) ne kadar gelişirse gelişsin, softmax normalizasyonu uzun bağlamlarda orta katman dikkatini seyreltir. 2) Re-ordering (Sıralama Hilesi): En yüksek skorlu dokümanları ortaya değil; sırasıyla [1. Doküman en başa, 2. Doküman en sona, 3. Doküman ikinci sıraya] gelecek şekilde alternatifli dizin. 3) LLMLingua & Context Compression: Küçük bir encoder ile gereksiz doldurma kelimeleri (stop words) silinerek token sayısı %50 sıkıştırılır; hem maliyet yarıya iner hem de modelin dikkati odaklanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Transformer Attention Degradation & Mitigation: 1) The Lost-in-the-Middle Phenomenon: Rigorous empirical studies (Liu et al.) demonstrate that LLM recall forms a distinct U-shaped curve across long contexts. Softmax attention distributions dilute over long token sequences, heavily favoring prefix and suffix tokens. 2) Attention-Aware Re-ordering: Place the 1 most authoritative chunk at the very beginning of the context block, the 2 chunk at the very end, and weaker chunks in the center. 3) Context Compression (LLMLingua): Employs a compact token pruner to strip low-entropy tokens, condensing context volume by 30-50% without factual loss.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25: Lost in the Middle Korumalı Bağlam Yeniden Sıralayıcı (Context Re-orderer)
import java.util.ArrayList;
import java.util.List;

public class ContextReorderingOptimizer {
    /**
     * En iyi dokümanları U-şeklindeki attention eğrisinin uçlarına yerleştirir.
     * Sıralama Mantığı: En iyi doküman başa (#1), ikinci en iyi doküman sona (#2)...
     */
    public static <T> List<T> reorderForAttention(List<T> sortedCandidates) {
        if (sortedCandidates.size() <= 2) return sortedCandidates;

        List<T> reordered = new ArrayList<>(sortedCandidates.size());
        int left = 0;
        int right = sortedCandidates.size() - 1;
        boolean toggle = true;

        while (left <= right) {
            if (toggle) {
                reordered.add(0, sortedCandidates.get(left++)); // Başa ekle
            } else {
                reordered.add(sortedCandidates.get(left++));     // Sona ekle
            }
            toggle = !toggle;
        }
        return reordered;
    }
}

void main() {
    List<String> topDocs = List.of("Doc1 (En İyi)", "Doc2 (İyi)", "Doc3 (Orta)", "Doc4 (Zayıf)");
    var optimized = ContextReorderingOptimizer.reorderForAttention(topDocs);

    IO.println("Attention Odaklı Bağlam Sıralaması:");
    optimized.forEach(IO::println);
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Mülakat Hatası: '128k context window var, RAG'den gelen 30 dokümanın hepsini prompt'a ekledim' demek mülakatın sonu olur. Fazla bağlam hem maliyeti katlar, hem yanıt süresini 5 saniyeye çıkarır, hem de 'Lost in the Middle' yüzünden modelin doğru cevabı ıskalamasına yol açar. İdeal bağlam 3 ila 5 filtrelenmiş parçadır.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Interview failure: Stuffing 30 retrieved chunks into prompt context under the naive assumption that 'the LLM will figure it out'. Excessive context inflates latency, drains budget, and guarantees retrieval failure due to attention dilution. Keep production context strictly capped to top 3-5 verified chunks.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- LLM'ler bağlamın başındaki ve sonundaki bilgileri çok iyi, ortasındakileri çok kötü hatırlar.
- En kritik kanıtlar bağlam blokunun en başına ve en sonuna yerleştirilmelidir.
- Bağlam sıkıştırma, token maliyetini %50 düşürürken modelin odaklanmasını sağlar.

🎯 Key Takeaways for the Interview

- Transformers exhibit a strong U-shaped attention bias, neglecting information buried in the middle.
- Structure retrieved context so that primary evidence brackets the start and finish of the prompt.
- Context compression slashes token bills while intensifying semantic attention on vital facts.

29

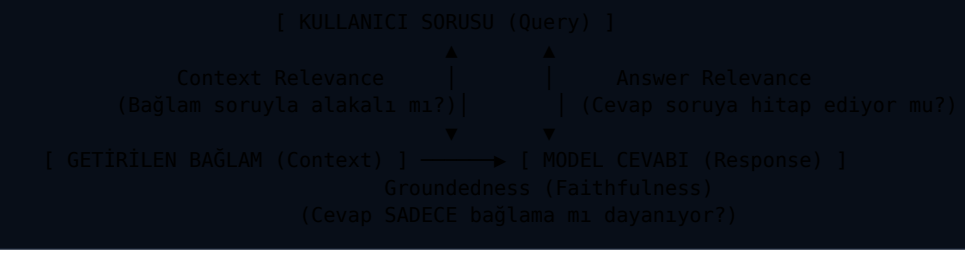
BÖLÜM: İLERI DÜZEY RAG (ADVANCED RAG) & BILGI GETIRME OPTIMIZASYONU | MODULE: ADVANCED RAG & INFORMATION RETRIEVAL OPTIMIZATION

RAG Değerlendirmesi: RAG Triad Metrikleri (Context Relevance, Groundedness, Answer Relevance)

The RAG Triad Evaluation: Context Relevance, Groundedness (Faithfulness) & Answer Relevance

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir mahkeme salonu düşünün: Hakim adil bir karar vermek için 3 şeye bakar: 1) Avukatın getirdiği delil olayla ilgili mi? (Context Relevance - Alakasız delil getirilirse dava uzar). 2) Karar SADECE masadaki delillere mi dayanıyor? (Groundedness / Faithfulness - Hakim kafasından uydurursa karar iptal edilir!). 3) Karar davacının sorduğu soruya net cevap veriyor mu? (Answer Relevance - Soru miras davası iken hakim trafik cezası kesemez!). RAG Triad bu 3 sorudur: Bu 3 bacağı sağlam kurarsanız RAG sisteminiz asla yalan söylemez!

🇬🇧 ENGLISH (ELI5)

Picture a high-stakes court trial evaluated across 3 non-negotiable legal standards: 1) Context Relevance: Are the exhibits submitted by the prosecutor actually pertinent to the crime? 2) Groundedness (Faithfulness): Does the verdict stem strictly from evidence admitted to the court docket, or did the jury hallucinate rumors? 3) Answer Relevance: Does the final judgment address the complaint filed by the plaintiff? The RAG Triad establishes these 3 mathematical checkpoints: score high on all three, and your AI is bulletproof!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

RAG Triad Metriklerinin Matematiksel ve Operasyonel Tanımları: 1) Context Relevance: Getirilen metin parçalarının ne kadarı soruyla doğrudan ilgilidir? Düşükse: Chunking boyutunuz çok büyüktür veya Vektör benzerlik eşiğiniz çok düşüktür. 2) Groundedness (Faithfulness / Sadakat): Modelin ürettiği cümlenin her bir iddiası verilen bağlam dokümanından kanıtlanabiliyor mu? Düşükse: Model halüsinasyon görmektedir; Temperature çok yüksektir veya prompt gevşektir. 3) Answer Relevance: Üretilen yanıt kullanıcının orijinal sorusunu tam olarak yanıtlıyor mu? Düşükse: Model gevezelik yapmıştır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Mathematical Foundations of the RAG Triad (TruLens / Ragas): 1) Context Relevance: Precision of retrieved chunks relative to the user query. Low score indicates noisy embeddings or poor chunk boundaries. 2) Groundedness (Faithfulness): Ratio of claims in the generated completion that can be logically inferred from the retrieved context. A faithfulness score <0.95 indicates active hallucination. 3) Answer Relevance: Semantic alignment between the completion and the original intent. Penalizes verbose tangents and evasive non-answers.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```
// Java 25: RAG Triad Değerlendirme Puan Kartı Modeli
public record RagTriadScorecard(
    double contextRelevance,   // 0.0 - 1.0 (Getirilen bağlamın isabeti)
    double groundednessScore,   // 0.0 - 1.0 (Halüsinasyonsuzluk / Sadakat)
    double answerRelevance      // 0.0 - 1.0 (Kullanıcı sorusuna uyum)
) {
    public boolean isProductionGrade() {
        return contextRelevance >= 0.80
            && groundednessScore >= 0.95   // MAANG standardı: %95 üzeri sadakat zorunludur!
            && answerRelevance >= 0.85;
    }

    public double computeCompositeScore() {
        return (contextRelevance + (groundednessScore * 2.0) + answerRelevance) / 4.0;
    }
}

void main() {
    var scorecard = new RagTriadScorecard(0.88, 0.98, 0.92);
    IO.println("RAG Triad Birleşik Kalite Skoru: " + scorecard.computeCompositeScore());
    IO.println("Canlıya Çıkış İzni: " + (scorecard.isProductionGrade() ? "✅ ONAYLANDI" : "❌ GÜVENLİK
SINIRI ALTINDA"));
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Kritik Metrik Hatası: Çoğu acemi ekip sadece 'Cevap güzel görünüyor mu' (Answer Relevance) diye bakar. Oysa cevap çok mantıklı ve akıcı görünse bile tamamen UYDURMA olabilir (Yüksek Answer Relevance, sıfır Groundedness)! Groundedness daima en öncelikli metriktir.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

The Fluent Hallucination Trap: Evaluating AI systems solely by whether answers 'look articulate'. An answer can sound authoritative and convincing while being 100% fabricated. Groundedness (Faithfulness) is the single most critical safety metric.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• RAG Triad; Context Relevance, Groundedness ve Answer Relevance ayaklarından oluşur.

• Groundedness (Sadakat), cevabın tamamen verilen dokümana dayanıp dayanmadığını ölçer.

• Kurumsal MAANG projelerinde Groundedness skoru %95'in altına düşemez.

🎯 Key Takeaways for the Interview

• The RAG Triad comprises Context Relevance, Groundedness, and Answer Relevance.

• Groundedness measures whether generated claims are strictly supported by context.

• Enterprise production gates mandate Groundedness scores of >=95% to prevent liability.

30

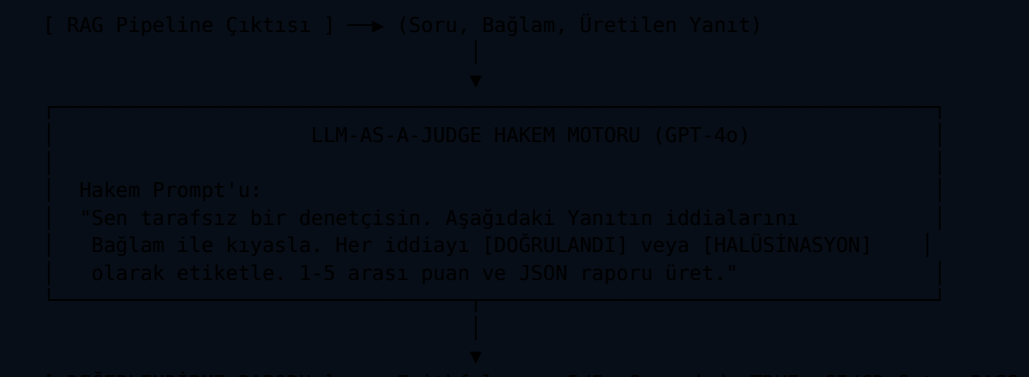
BÖLÜM: İLERI DÜZEY RAG (ADVANCED RAG) & BILGI GETIRME OPTIMIZASYONU | MODULE: ADVANCED RAG & INFORMATION RETRIEVAL OPTIMIZATION

RAG Triad Otomasyonu: RAGAS Metodolojisi & LLM-as-a-Judge Mimarisi

Automated RAG Evaluation: RAGAS Framework & Implementing LLM-as-a-Judge

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir okulda 10.000 öğrencinin yazdığı kompozisyonları tek bir öğretmenin sabahlara kadar tek tek okuyup puanlaması imkansızdır (İnsanla değerlendirme çok yavaştır). Bunun yerine kıdemli bir başöğretmene (LLM-as-a-Judge) bir puanlama anahtarı verirsiniz: 'Şu 3 kurala bak, 1 ile 5 arasında not ver, nerede hata varsa altına yaz.' Başöğretmen her gece tüm ödevleri inceler ve sabah not çizelgesini masanıza koyar! LLM-as-a-Judge, yapay zekayı yapay zekaya denetleterek sistem kalitesini her kod gönderildiğinde (CI/CD) otomatik ölçmektir!

🇬🇧 ENGLISH (ELI5)

Evaluating 100,000 production customer responses cannot be done manually by human reviewers (human annotation is slow and unscalable). Instead, you employ an impartial Senior Inspector model (LLM-as-a-Judge) armed with an invariant grading rubric: 'Deconstruct the response into individual claims. Cross-verify each claim against the retrieved snippet. Output a structured JSON audit with a score from 1 to 5.' LLM-as-a-Judge runs automatically inside your CI/CD test suite on every pull request!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

LLM-as-a-Judge Mimarisi ve İstatistiki Yanlılıklar: 1) Hakem Önyargıları (Biases): a) Position Bias (İlk seçeneği üstün tutma). b) Verbosity Bias (Uzun ve süslü yanıtları daha kaliteli sanma). c) Self-Enhancement Bias (Modelin kendi ürettiği metinlere daha yüksek puan vermesi). 2) İki Taraflı Değerlendirme (Pairwise Evaluation with Swap): Seçeneklerin sırası ters çevrilerek hakeme iki kez sorulur; sadece iki seferde de aynı karar çıkarsa puan geçerli sayılır. 3) CI/CD Entegrasyonu: GitHub Actions üzerinde her PR açıldığında 50 adet altın soru (Gold Evaluation Dataset) test edilir; sadakat skoru %95'in altına düşerse PR kilitlenir!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

LLM-as-a-Judge Vulnerabilities & CI/CD Gating: 1) Cognitive Judge Biases: a) Verbosity Bias: Models favor longer, eloquently worded responses regardless of precision. b) Position Bias: Predisposition to score Option A higher than Option B in pairwise tests. Countermeasure: execute bidirectional swaps (A/B and B/A). c) Self-Preference Bias: GPT-4 exhibits affinity for GPT-generated prose over Claude or Llama. 2) CI/CD Automation: Run automated evaluation pipelines against a fixed 100-question Golden Dataset on every pull request. If regression drops Faithfulness below 95%, the deployment pipeline aborts.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & Spring AI: LLM-as-a-Judge ile Otomatik RAG Hakem Servisi
import org.springframework.ai.chat.client.ChatClient;

public class RagJudgeEvaluator {
    private final ChatClient judgeChatClient;

    public record EvaluationVerdict(double faithfulnessScore, String critique, boolean passed) {}

    public RagJudgeEvaluator(ChatClient.Builder builder) {
        this.judgeChatClient = builder.build();
    }

    public EvaluationVerdict auditRagResponse(String query, String context, String generatedAnswer) {
        return judgeChatClient.prompt()
            .system("""
                Sen MAANG seviyesinde katı bir yapay zeka denetçisisin.
                Görevin: Verilen Yanıt'ın SADECE ve SADECE Bağlam'daki bilgilere dayanıp dayanmadığını
doğrulamak.
                Eğer yanıtta bağlamda geçmeyen tek bir iddia bile varsa faithfulnessScore < 0.50
olmalı ve passed = false dönmelidir.
                """)
            .user(u -> u.text("""
                SORU: {query}
                BAĞLAM: {context}
                ÜRETİLEN YANIT: {answer}
                """)
                .param("query", query)
                .param("context", context)
                .param("answer", generatedAnswer))
            .call()
            .entity(EvaluationVerdict.class);
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Hakem Modeli Kuralı: Küçük modeller (GPT-4o-mini) karmaşık mantıksal denetimlerde hakemlik yapamaz; ince halüsinasyonları gözden kaçırır! Hakem modeli olarak DAİMA en zeki amiral gemisi model (GPT-4o veya o1) kullanılmalıdır.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Judge capacity requirement: Never use miniature tier models (e.g. gpt-4o-mini) as judges. Sub-parameter models lack the cognitive reasoning required to identify subtle factual inconsistencies. The Judge model must strictly be Tier-1 (GPT-4o, Claude 3.5 Sonnet, or o1).

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- LLM-as-a-Judge, yüz binlerce RAG çıktısını otomatik ve ölçeklenebilir biçimde denetler.
- Hakem önyargılarını (Verbosity ve Position bias) kırmak için çift yönlü testler şarttır.
- Altın test veri setleri (Golden Datasets) ile CI/CD pipeline'larında regresyonlar engellenir.

🎯 Key Takeaways for the Interview

- LLM-as-a-Judge scales quality assurance across millions of RAG interactions cost-effectively.
- Counteract verbosity and position biases via pairwise order swaps.
- Embedding evaluation suites in CI/CD ensures new prompts do not cause quality regressions.